

COMPANION
to

ChatGPT Teaches TikZ

Chapter 21: Creating Your Own TikZ Keys (A)

Contents

21.1	What This Companion Adds	2
21.2	A Custom-Key Vocabulary	3
21.3	Parameters versus Named Keys	3
21.4	A Key that Executes Code	4
21.5	Storing a Value	4
21.6	Initial Values and Retrieval	5
21.7	Organizing Keys in a Directory	5
21.8	Choice Keys	6
21.9	Separating Interface and Implementation	7
21.10	Worked Project: A Configurable Vertex	7
21.11	Debugging Workshop	9
21.12	Exercises	10
21.13	Solutions	11
21.14	ChatGPT Workshop	12
21.15	Final Checklist	12

21.1 What This Companion Adds

Chapter 21 introduced custom TikZ keys as a named interface to reusable components. This Companion distinguishes four jobs: executing code, storing a value, providing an initial value, and restricting a setting to named choices.

Design

Expose the choices a user should make. Keep the lower-level TikZ options that implement those choices inside the component.

21.2 A Custom-Key Vocabulary

Handler or form	Purpose
<code>/.code=</code>	Executes code when the key is used.
<code>/.store in=</code>	Stores the supplied token list in a macro.
<code>/.estore in=</code>	Stores the expanded value in a macro.
<code>/.initial=</code>	Gives a key an initial stored value.
<code>/.default=</code>	Supplies a value when a key is used without one.
<code>/.is choice</code>	Restricts a key to named alternatives.
<code>/.cd</code>	Changes the current key directory.
<code>\pgfkeysvalueof</code>	Retrieves the stored value of a key path.

21.3 Parameters versus Named Keys

The call

`component={2cm/blue!15/$A$}`

requires the reader to remember three positions. Named keys document themselves.

```

component radius=2cm,
component fill=blue!15,
component label={A$}

```

Values may be written in any order, and each name states its purpose.

21.4 A Key that Executes Code

```

\tikzset{
  announce/.code={\def\announcement{#1}}
}

```

Using `announce=Ready` executes the attached definition with `Ready` substituted for `#1`.

```

\begin{tikzpicture}[announce=Ready]
  \node at (0,0) {\announcement};
\end{tikzpicture}

```

Use `/.code` when the key must perform an action. For a value that later commands merely read, a storage handler is simpler.

21.5 Storing a Value

```

\tikzset{
  diagram radius/.store in=\diagramradius,
  diagram radius=1cm
}

```

The second line initializes the storage macro through the key itself.

```

\begin{tikzpicture}[diagram radius=1.5cm]
  \draw (0,0) circle (\diagramradius);
\end{tikzpicture}

```

Choose a distinctive macro name to avoid collisions with other packages or components.

21.6 Initial Values and Retrieval

```
\tikzset{
  diagram radius/.initial=1cm
}
```

Retrieve the value by its complete path.

```
\draw (0,0) circle
  (\pgfkeysvalueof{/tikz/diagram radius});
```

A picture can replace the initial value.

```
\begin{tikzpicture}[diagram radius=1.6cm]
  ...
\end{tikzpicture}
```

21.7 Organizing Keys in a Directory

Related keys belong in one directory.

```
\tikzset{
  configurable vertex/.cd,
  size/.initial=8mm,
  fill color/.initial=white,
  line color/.initial=black
}
```

The full path of the first setting includes both directories.

```
/tikz/configurable vertex/size
```

A directory prevents general names such as `size` from colliding with unrelated keys.

21.8 Choice Keys

```
\tikzset{
  orientation/.is choice,
  orientation/horizontal/.style={x=1cm,y=1cm},
  orientation/vertical/.style={x=1cm,y=-1cm},
  orientation/.default=horizontal
}
```

The user chooses one of the named values.

```
orientation=horizontal
orientation=vertical
```

Any other value produces an error rather than a silently inconsistent drawing.

Common Mistake

Use a choice key only for a fixed vocabulary. A continuously varying dimension such as a radius should remain an ordinary value key.

21.9 Separating Interface and Implementation

The keys state what may vary. A style reads their values and translates them into ordinary TikZ options.

```
\tikzset{
  /tikz/configured vertex/.style={
    draw,circle,
    minimum size=\pgfkeysvalueof{
      /tikz/configurable vertex/size},
    fill=\pgfkeysvalueof{
      /tikz/configurable vertex/fill color},
    draw=\pgfkeysvalueof{
      /tikz/configurable vertex/line color}
  }
}
```

The user need not know that `minimum size`, `fill`, and `draw` implement the three settings.

21.10 Worked Project: A Configurable Vertex

21.10.1 Stage 1: Define the Interface

```
\tikzset{
  configurable vertex/.cd,
  size/.initial=8mm,
  fill color/.initial=white,
  line color/.initial=black
}
```

21.10.2 Stage 2: Define the Drawing Style

```
\tikzset{
  /tikz/configured vertex/.style={
    draw,circle,
    minimum size=\pgfkeysvalueof{
      /tikz/configurable vertex/size},
    fill=\pgfkeysvalueof{
      /tikz/configurable vertex/fill color},
    draw=\pgfkeysvalueof{
      /tikz/configurable vertex/line color}
  }
}
```


21.10.3 The Complete Picture

```
\begin{tikzpicture}
  \node[configured vertex] at (0,0) {$A$};

  \begin{scope}[
    configurable vertex/size=11mm,
    configurable vertex/fill color=blue!15,
    configurable vertex/line color=blue!60!black
  ]
    \node[configured vertex] at (2.5,0) {$B$};
  \end{scope}

  \node[configured vertex] at (5,0) {$C$};
\end{tikzpicture}
```

See



Design

The key directory is the interface. The configured style is the implementation. The scope makes one temporary configuration local.

21.11 Debugging Workshop

21.11.1 A Stored Macro Is Undefined

Initialize the key before a drawing command reads its storage macro.

21.11.2 Retrieval Returns the Wrong Key

Use the complete path in `\pgfkeysvalueof`, including the `/tikz` directory and component directory.

21.11.3 A Key Is Defined in the Wrong Directory

After using `/.cd`, define unrelated keys with an absolute path or return explicitly to `/tikz/.cd`.

21.11.4 An Invalid Choice Is Accepted

Confirm that the parent key uses `/.is choice` and that each allowed value has its own subkey.

21.12 Exercises

1. Create a code key that stores a short label.
2. Create a value key for a circle radius.
3. Give the radius a useful initial value.
4. Retrieve the value with its complete key path.
5. Place three related keys in one directory.
6. Create a three-way choice key.
7. Build a style that reads two custom key values.
8. Replace a three-argument component interface with named keys.

21.13 Solutions

21.13.1 Exercises 1–4

```
\tikzset{
  remember label/.code={\def\savedlabel{#1}},
  object radius/.initial=1cm
}
\draw (0,0) circle
  (\pgfkeysvalueof{/tikz/object radius});
```

21.13.2 Exercises 5–7

```
\tikzset{
  layout/.is choice,
  layout/compact/.style={x=.8cm,y=.8cm},
  layout/normal/.style={x=1cm,y=1cm},
  layout/spacious/.style={x=1.3cm,y=1.3cm}
}
```

Exercise 7 retrieves the values by full path inside an ordinary style. Exercise 8 gives each former argument a semantic key name and an initial value.

21.14 ChatGPT Workshop

Ask ChatGPT to review the public interface separately from its implementation.

1. Replace these positional parameters with semantic custom keys.
2. Choose the correct handler for each key and explain the choice.
3. Add safe initial values and distinctive storage macro names.
4. Diagnose every incorrect or incomplete key path in this component.
5. Design a choice key for this fixed set of meaningful alternatives.

21.15 Final Checklist

Before continuing to Chapter 22, check that you can

1. distinguish code, storage, initial-value, and choice handlers;
2. create a key that executes code;
3. store and retrieve a configurable value;
4. organize related keys in a directory;
5. use complete key paths reliably;
6. restrict a setting to named choices; and
7. separate a component's interface from its implementation.